Solution 1

Solution 1 (a)

Python Code

```python
import numpy as np
import matplotlib.pyplot as plt

# set random seed
np.random.seed(42)

# define pi, mu, sigma
pi = np.array([0.1, 0.2, 0.5, 0.2])
mu = [np.array([7, 6]), np.array([0, 0]), np.array([-1, 4]), np.array([5, -2])]
sigma = [np.array([[1, 0], [0, 25]]), np.array([[1, 0], [0, 1]]),
    np.array([[9, 1], [1, 1]]), np.array([[16, -6], [-6, 4]])]

# generate 400 random points from the mixture model
N = 400
z_labels = np.random.choice(len(pi), size=N, p=pi)

# make containers for X data ,true labels data
X_data = []
true_labels = []

# sample from the specific Gaussian for each component
for i in range(4):
    # Count how many points belong to component i
    n_i = np.sum(z_labels == i)
    if n_i > 0:
        # Sample n_i points from N(mu_i, Sigma_i)
        pts = np.random.multivariate_normal(mu[i], sigma[i], n_i)
        X_data.append(pts)
        true_labels.append(np.full(n_i, i))

# massage data for plots
X = np.vstack(X_data)
y = np.concatenate(true_labels)

# plot points, using colors to distinguish each of the components
plt.figure(figsize=(8, 6))
colors = ['red', 'green', 'blue', 'orange']
labels = ['$\pi_1 N(\mu_1, \Sigma_1)$', '$\pi_2 N(\mu_2, \Sigma_2)$',
    '$\pi_3 N(\mu_3, \Sigma_3)$', '$\pi_4 N(\mu_4, \Sigma_4)$']

for i in range(4):
    plt.scatter(X[y == i, 0], X[y == i, 1],
    c=colors[i], label=labels[i], alpha=0.6, s=20)

plt.title('Q1(a): Generated Data from 4-Component GMM')
plt.xlabel('X1')
plt.ylabel('X2')
plt.xlim(-15,18)
plt.ylim(-7,18)
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('q1a.png') # Save for LaTeX inclusion
```

Solution 1

Solution 1 (a)

Plot

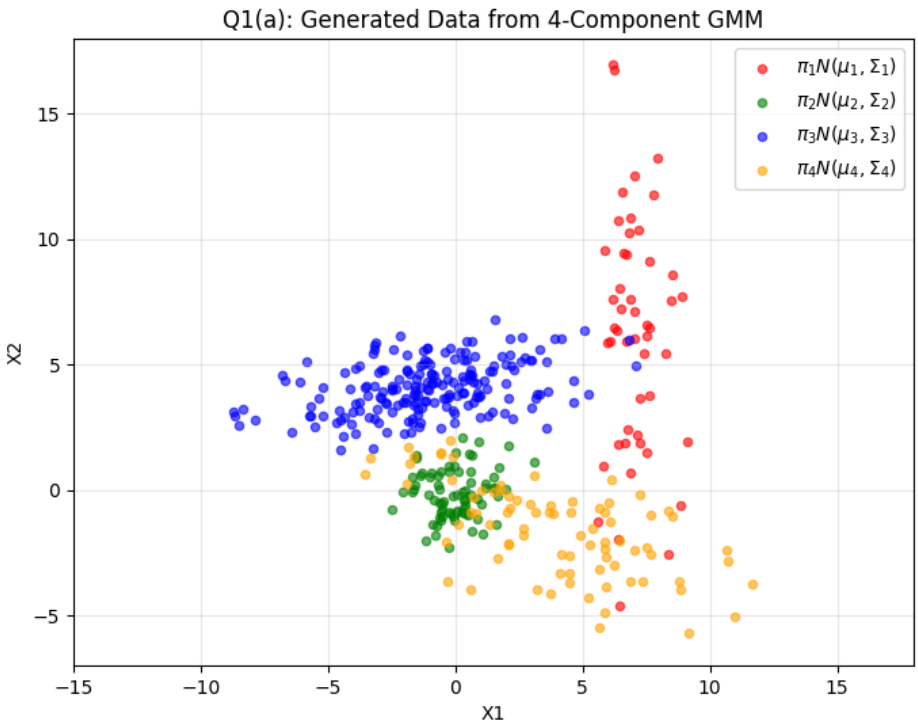

Figure 1: Visualization of the 400 random points generated from the mixture model.

Solution 1

Solution 1 (b)

Python Code

```python
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse

# set random seed
np.random.seed(42)

# define pi, mu, sigma
pi = np.array([0.1, 0.2, 0.5, 0.2])
mu = [np.array([7, 6]), np.array([0, 0]), np.array([-1, 4]), np.array([5, -2])]
sigma = [np.array([[1, 0], [0, 25]]), np.array([[1, 0], [0, 1]]),
    np.array([[9, 1], [1, 1]]), np.array([[16, -6], [-6, 4]])]

# generate 400 random points from the mixture model
N = 400
z_labels = np.random.choice(len(pi), size=N, p=pi)

# make containers for X data ,true labels data
X_data = []
true_labels = []

# sample from the specific Gaussian for each component
for i in range(4):
    # Count how many points belong to component i
    n_i = np.sum(z_labels == i)
    if n_i > 0:
        # Sample n_i points from N(mu_i, Sigma_i)
        pts = np.random.multivariate_normal(mu[i], sigma[i], n_i)
        X_data.append(pts)
        true_labels.append(np.full(n_i, i))

# massage data for plots
X = np.vstack(X_data)
y = np.concatenate(true_labels)

def plot_ellipse(mu, sigma, ax, color):
    # calc eigen values/vectors
    vals, vecs = np.linalg.eigh(sigma)

    # Sort eigenvalues/vectors large -> small
    order = vals.argsort()[::-1]
    vals = vals[order]
    vecs = vecs[:, order]

    # calc angle
    theta = np.degrees(np.arctan2(*vecs[:,0][::-1]))

    # calc width and height based on eigenvalues
    width, height = 2.0 * np.sqrt(3.0 * vals)

    # create ellipse
    ell = Ellipse(xy=mu, width=width, height=height, angle=theta,
                  edgecolor=color, facecolor='none', lw=2, label='_nolegend_')
    ax.add_artist(ell)

# make plot
fig, ax = plt.subplots(figsize=(8, 6))
colors = ['red', 'green', 'blue', 'orange']
labels = ['$\pi_1 N(\mu_1, \Sigma_1)$', '$\pi_2 N(\mu_2, \Sigma_2)$',
    '$\pi_3 N(\mu_3, \Sigma_3)$', '$\pi_4 N(\mu_4, \Sigma_4)$']

for i in range(4):
    plt.scatter(X[y == i, 0], X[y == i, 1], c=colors[i],
```

```
        label=labels[i], alpha=0.6, s=20)
    plot_ellipse(mu[i], sigma[i], ax, color=colors[i])
    # mark center of ellipse
    ax.scatter(mu[i][0], mu[i][1], c='black', marker='x', s=50)

plt.title('Q1(b): Data Points with Covariance Ellipses')
plt.xlabel('X1')
plt.ylabel('X2')
plt.legend()
plt.grid(True, alpha=0.3)
plt.axis('equal')
plt.savefig('q1b.png')
```

Plot

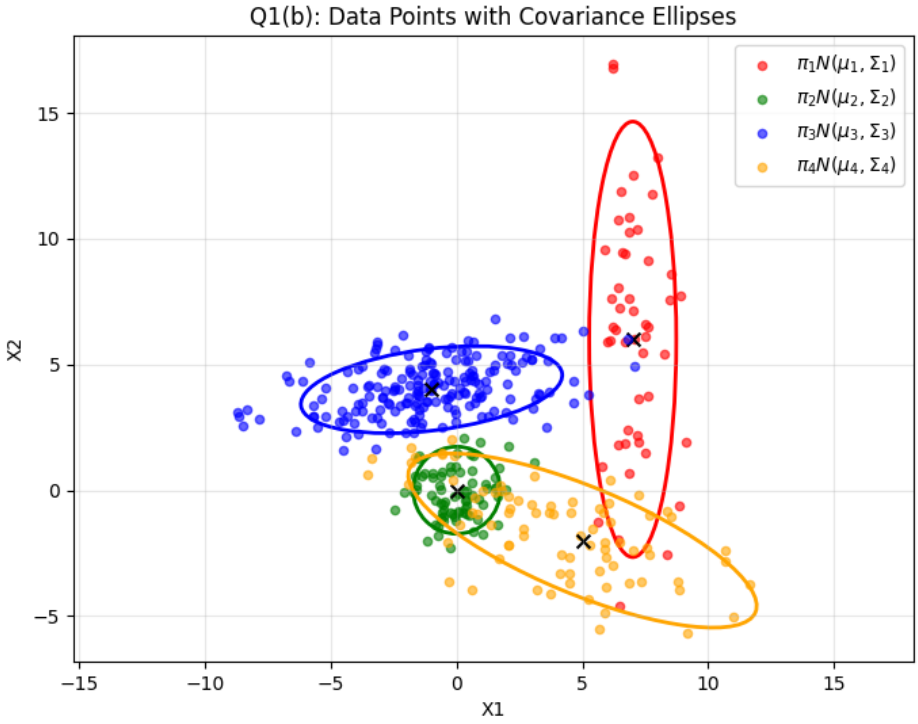

Figure 2: Visualization of the 400 random points (black) overlaid with red ellipses representing the 99% confidence intervals of the four Gaussian components.

Solution 1

Solution 1 (c)

Plot

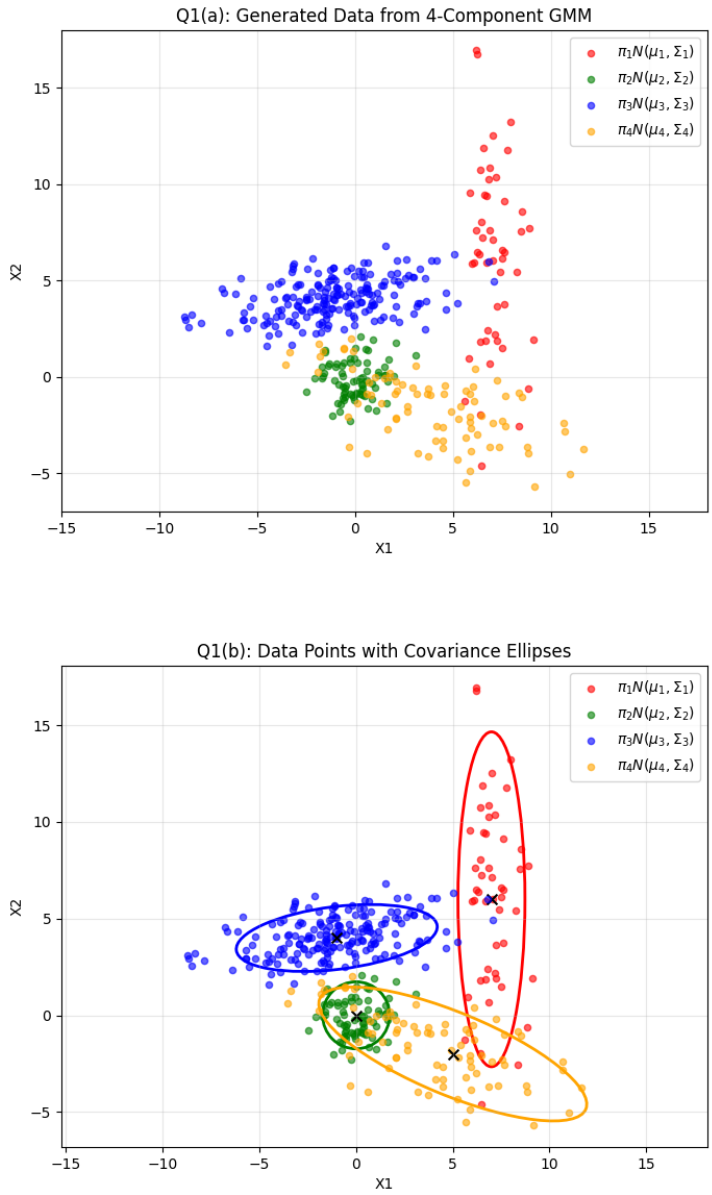

Figure 3: Comparison of plots from *part (a)* and *part (b)*. The *2nd Cluster* (Green, $\mu_2 = [0, 0]$) is the small cluster that is partially contained by the *4th Cluster* (Orange, $\mu_4 = [5, -2]$)

Solution 1

Solution 1 (d)

Python Code

```python
from sklearn.mixture import GaussianMixture

# initialize and fit the GMM
gmm = GaussianMixture(n_components=4, covariance_type='full', random_state=42)
gmm.fit(X)

fig2, ax2 = plt.subplots(figsize=(10, 8))

# plot orginal data
ax2.scatter(X[:, 0], X[:, 1], c='gray', s=15, alpha=0.2, label='Observed Data')

# plot true
for i in range(4):
    plot_ellipse(mu[i], sigma[i], ax2, color=colors[i], linestyle='solid',
        linewidth=1.5)
    # add filler for legend
    ax2.scatter([], [], c=colors[i], label=f'True Comp {i+1}')

# plot model
for i in range(4):
    learned_mean = gmm.means_[i]
    learned_cov = gmm.covariances_[i]

    # plot learned ellipse
    plot_ellipse(learned_mean, learned_cov, ax2, color='black', linestyle='--',
        linewidth=2.5)

    # mark learned centers
    label = 'Learned Parameters' if i == 0 else ""
    ax2.scatter(learned_mean[0], learned_mean[1], c='black',
                marker='+', s=120, lw=2, label=label)

ax2.set_title('Q1(d): Comparison of Learned (Black/Dashed) vs True (Color/Solid)',
    fontsize=14)
ax2.set_xlabel('X1')
ax2.set_ylabel('X2')
ax2.axis('equal')
ax2.grid(True, alpha=0.3)
ax2.legend(loc='upper right')
plt.savefig('q1d.png')
```

Solution 1

Solution 1 (d)

Plot

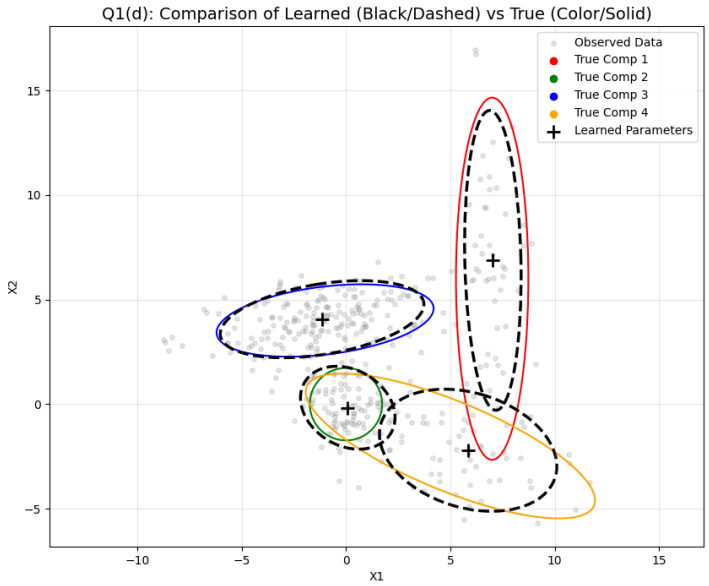

Figure 4: Visualization of the ellipses learned by the EM algorithm (blue crosses denote learned means) overlaid on the original data.

∴ The Gaussians learned by the EM algorithm are close to the true generative parameters, with only minor deviations due to noise and sample size.

Solution 1

Solution 1 (e)

Python Code

```python
from sklearn.mixture import GaussianMixture

# initialize and fit the GMM
gmm = GaussianMixture(n_components=4, covariance_type='full', random_state=42)
gmm.fit(X)

# make predictions
y_pred = gmm.predict(X)

# plot predictions
plt.figure(figsize=(8, 6))

# define colors for predicted data
colors_pred = ['purple', 'cyan', 'lime', 'magenta']
for k in range(4):
    # select points predicted to belong to cluster k
    mask = (y_pred == k)
    plt.scatter(X[mask, 0], X[mask, 1], s=20, label=f'Cluster {k}', alpha=0.6)

plt.title('Q1(e): Data Colored by GMM Hard Clustering')
plt.xlabel('X1')
plt.ylabel('X2')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('q1e.png')
plt.show()
```

Solution 1

Solution 1 (e)

Plot

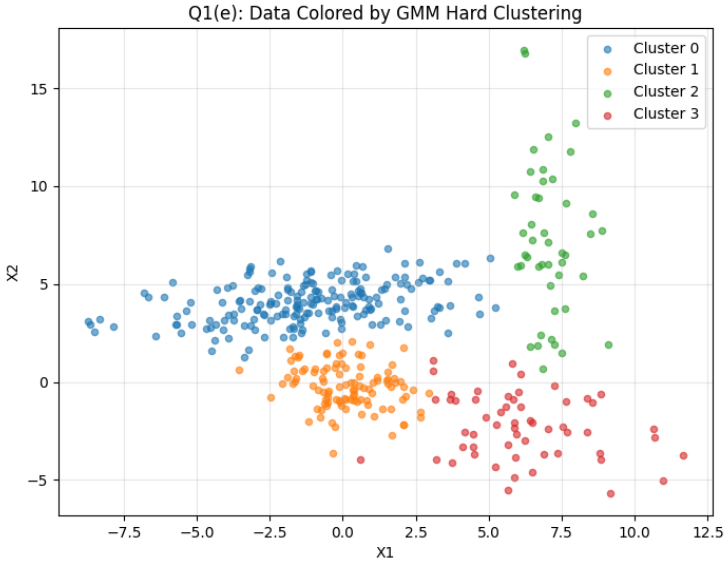

Figure 5: Visualization of the data points colored by their assigned cluster (Hard Assignment).

Plot Comments

The clustering is not perfect, but it captures the general structure of the data well.

The discrepancies are understandable since this happens where there is overlap.

Solution 1

Solution 1 (f)

Python Code

```python
import warnings
from sklearn.mixture import GaussianMixture

# set iteration snapshots
iterations = [1, 5, 10, 50]

plt.figure(figsize=(15, 10))

for idx, n_iter in enumerate(iterations):
    # initialize GMM
    gmm = GaussianMixture(n_components=4,
                          covariance_type='full',
                          max_iter=n_iter,
                          random_state=42,
                          init_params='random')

    # suppress warnings
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        gmm.fit(X)

    # plot subplot
    ax = plt.subplot(2, 2, idx+1)
    ax.scatter(X[:, 0], X[:, 1], c='k', s=5, alpha=0.2)

    # plot learned ellipses
    for i in range(4):
        plot_ellipse(gmm.means_[i], gmm.covariances_[i], ax)
        ax.scatter(gmm.means_[i][0], gmm.means_[i][1], c='blue', marker='+')

    ax.set_title(f'Iteration {n_iter}')
    ax.axis('equal')

plt.tight_layout()
plt.savefig('q1f_evolution.png')
plt.show()
```

Solution 1

Solution 1 (f)

Plot

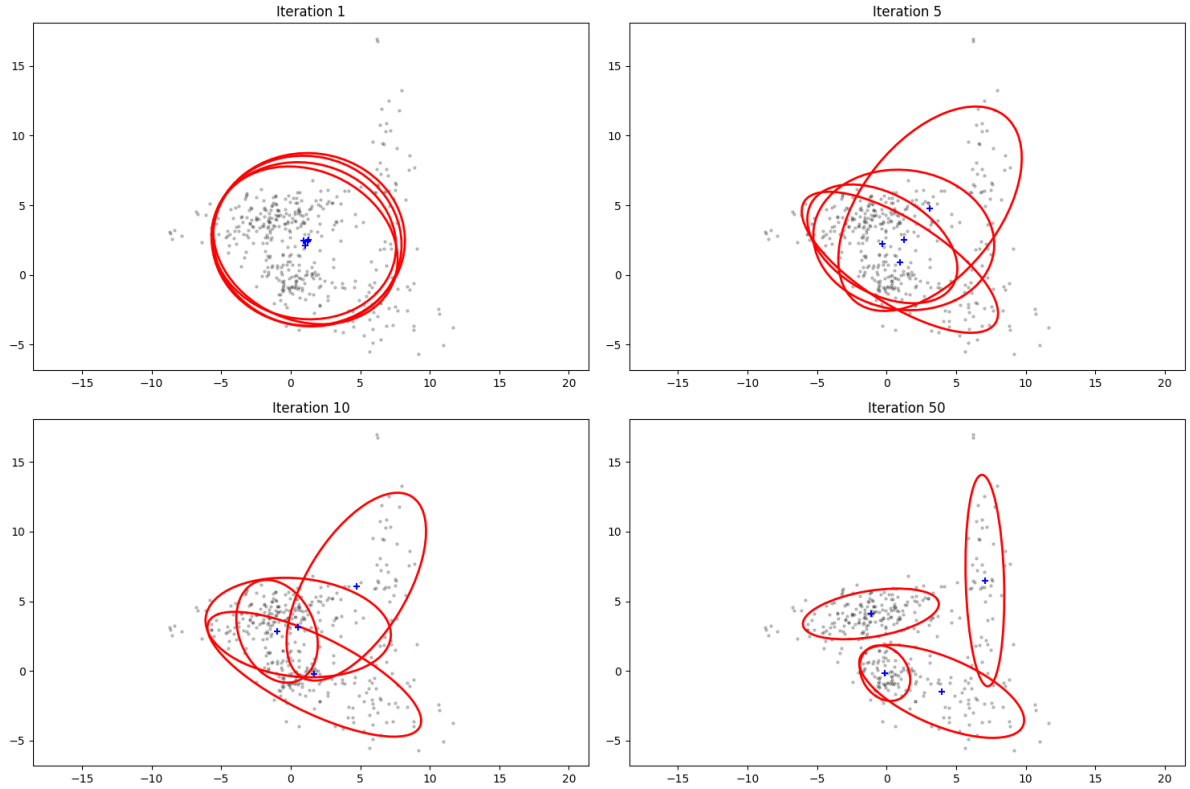

Figure 6: Evolution of the EM algorithm at iterations 1, 5, 10, and 50.

Solution 2

Solution 2 (a)

Python Code

```python
import numpy as np
import matplotlib.pyplot as plt

# load the samples.txt
data = np.loadtxt('samples.txt')

# plot histogram
plt.figure(figsize=(8, 6))
plt.hist(data, bins=20, density=True, color='skyblue', edgecolor='black', alpha=0.7)
plt.title('Q2(a): Histogram of 1,000 Samples (20 Bins)')
plt.xlabel('Value (x)')
plt.ylabel('Density')
plt.grid(True, alpha=0.3)
plt.xlim(0, 12)
plt.savefig('q2a.png')
```

Plot

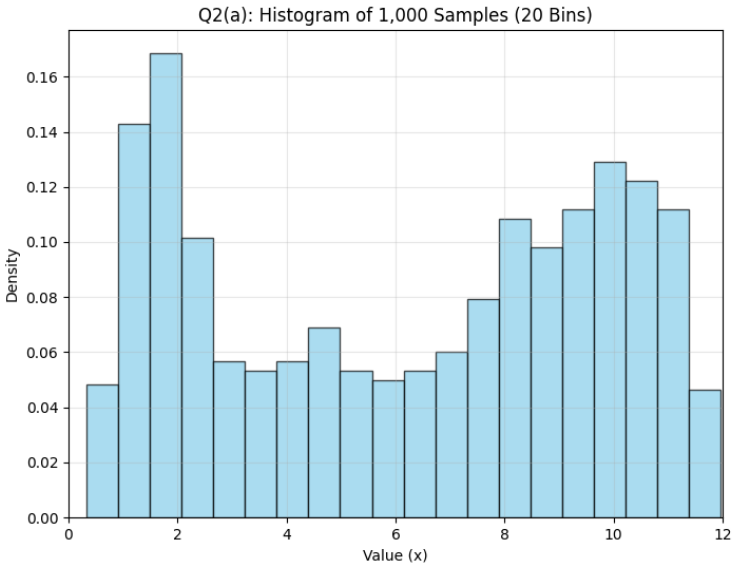

Figure 7: Histogram of the 1,000 samples using 20 bins.

Solution 2

Solution 2 (b)

Python Code

```python
from sklearn.neighbors import KernelDensity
import matplotlib.pyplot as plt
import numpy as np

# load samples.txt
data = np.loadtxt('samples.txt')

# select first 100 samples
X_train = data[:100].reshape(-1, 1)

X_plot = np.linspace(0, 12, 1000)[:, np.newaxis]

# bandwidths to test
bandwidths = [0.25, 0.50, 0.75, 1.0]

plt.figure(figsize=(12, 10))

for i, bw in enumerate(bandwidths):
    # initialize and fit KDE
    kde = KernelDensity(kernel='gaussian', bandwidth=bw)
    kde.fit(X_train)

    # convert log-density
    log_dens = kde.score_samples(X_plot)

    # plot
    ax = plt.subplot(2, 2, i+1)
    ax.fill(X_plot, np.exp(log_dens), fc='skyblue')
    ax.plot(X_plot, np.exp(log_dens), color='navy', lw=2)

    ax.set_title(f'Bandwidth h = {bw}')
    ax.set_ylim(0.004, 0.16)
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('q2b_combined.png')
```

Solution 2

Solution 2 (b)

Plot

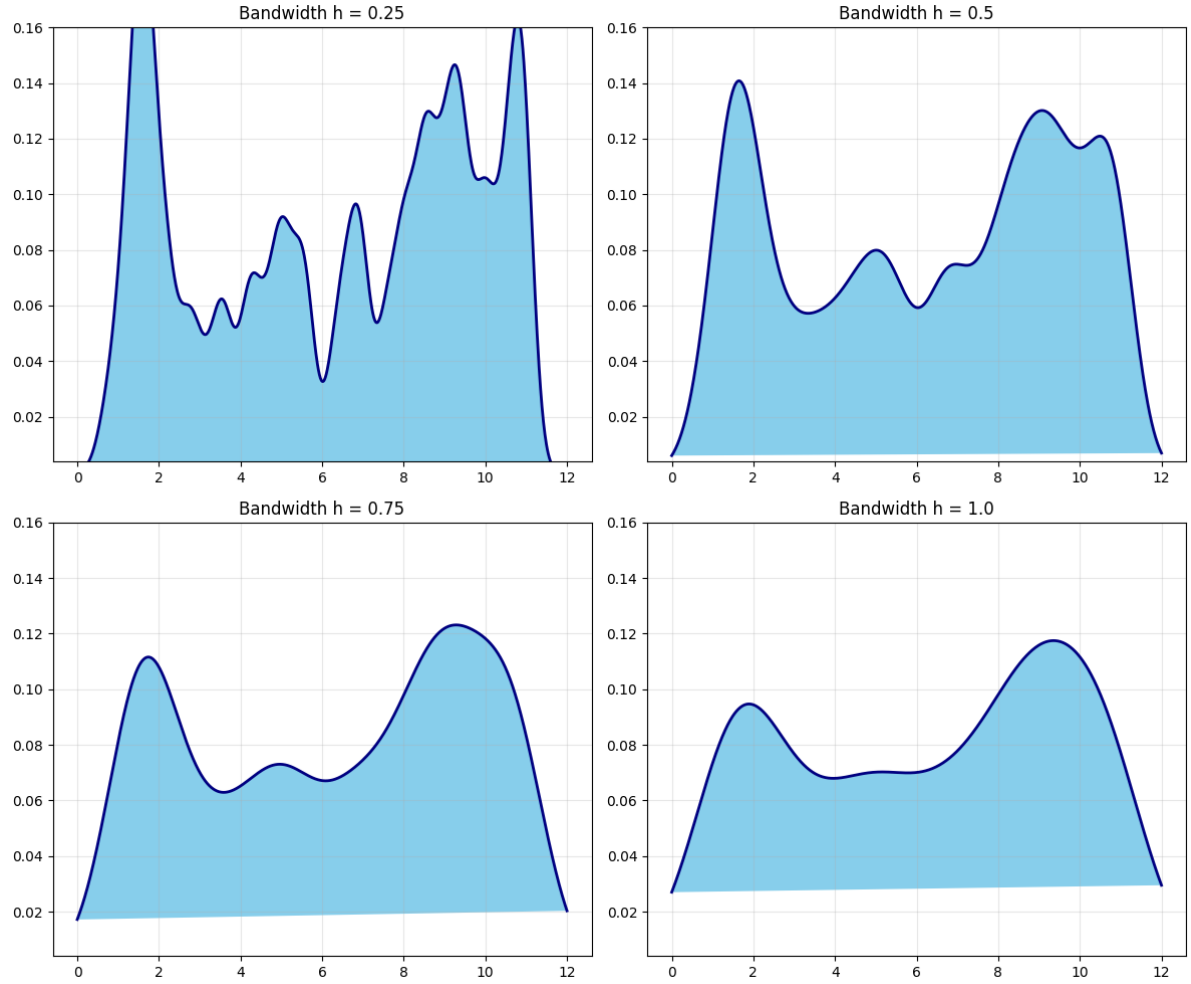

Figure 8: KDE estimates using the first 100 samples with varying bandwidths.

Plot Comments

As the bandwidth increases, the estimated density function becomes smoother and flatter. The narrow, peaks and valleys become less pronounced as bandwidth is increased. The bandwidth $h = 0.50$ appears to yield the model closest to the original f.

Solution 2

Solution 2 (c)

Python Code

```python
from sklearn.neighbors import KernelDensity
import matplotlib.pyplot as plt
import numpy as np

# load samples.txt
data = np.loadtxt('samples.txt')

# select all 1000 samples
X_train = data.reshape(-1, 1)

X_plot = np.linspace(0, 12, 1000)[:, np.newaxis]

# bandwidths to test
bandwidths = [0.25, 0.50, 0.75, 1.0]

plt.figure(figsize=(12, 10))

for i, bw in enumerate(bandwidths):
    # initialize and fit KDE
    kde = KernelDensity(kernel='gaussian', bandwidth=bw)
    kde.fit(X_train)

    # convert log-density
    log_dens = kde.score_samples(X_plot)

    # plot
    ax = plt.subplot(2, 2, i+1)
    ax.fill(X_plot, np.exp(log_dens), fc='skyblue')
    ax.plot(X_plot, np.exp(log_dens), color='navy', lw=2)

    ax.set_title(f'Bandwidth h = {bw}')
    ax.set_ylim(0.004, 0.16)
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('q2c_combined.png')
plt.show()
```

Solution 2

Solution 2 (c)

Plot

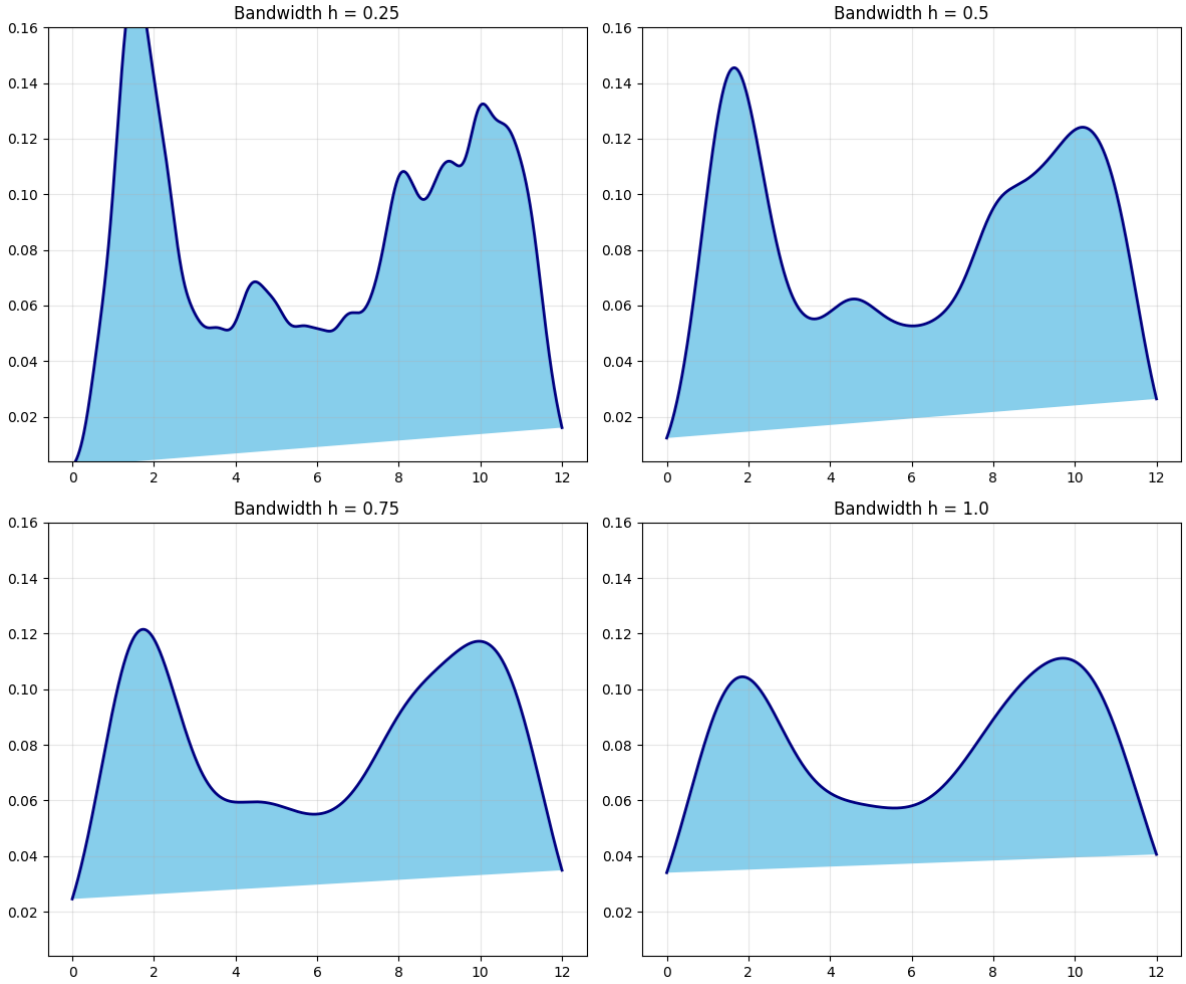

Figure 9: KDE estimates using all 1000 samples.